

CS 3510 Algorithms: Arithmetic and Fast Fourier Transform

Aaron Hillegass
Georgia Tech

Unsigned Ints

n bits? Store $0..2^n - 1$

Storing x ? Need $n = O(\log x)$ bits

Addition

```
  1000101
+1110111
-----
 10111100
```

Your computer sums two 64-bit in a single clock cycle.

What about 2048-bit numbers?

Generally for n -bit numbers, $T(n) \in O(n)$

Doubling and Halving

$$9 \times 2$$

Shift left, fill with zeros:

$$1001 = 9$$

$$10010 = 18$$

$$\left\lfloor \frac{9}{2} \right\rfloor$$

Shift right:

$$1001 = 9$$

$$100 = 4$$

Your computer shifts a 64-bit number by k in a single clock cycle.

Multiplication: Naive

$$13 \times 11 = ?$$

Left-shift and accumulate:

$$\begin{array}{r}
 1011 = 11 \\
 \times 1101 = 13 \\
 \hline
 1011 \\
 0000 \\
 1011 \\
 1011 \\
 \hline
 10001111 = 143
 \end{array}$$

For n -bit numbers with $O(n)$ addition, this multiplication is $O(n^2)$.

Multiplication

Iterative

```

1 define mult( $x, y$ ):
2   y_stack := bits of  $y$  (MSB at top)
3   sum := 0
4   while bit := y_stack.pop():
5     | sum :=  $2 \times$  sum
6     | if bit:
7       | sum := sum +  $x$ 
8   return sum

```

Recursive

```

1 define mult2( $x, y$ ):
2   if  $y$  is zero:
3     | return zero
4    $z :=$  mult2( $x, \lfloor \frac{y}{2} \rfloor$ )
5   if  $y$  is even:
6     | return  $2z$ 
7   else
8     | return  $2z + x$ 

```

Still $O(n^2)$.

Multiply: Divide-and-Conquer 1

Multiplying x and y , divide each into two halves:

$$x = 2^{\frac{n}{2}} x_L + x_R \text{ and } y = 2^{\frac{n}{2}} y_L + y_R$$

Note:

$$xy = (2^{\frac{n}{2}} x_L + x_R)(2^{\frac{n}{2}} y_L + y_R) = 2^n x_L y_L + 2^{\frac{n}{2}} (x_L y_R + x_R y_L) + x_R y_R$$

Recurrence: $T(n) = 4T(\frac{n}{2}) + O(n)$, so $a = 4$, $b = 2$, $d = 1$

$$d < \log_b a \text{ so } O(n^{\log_b a}) = O(n^2)$$

Is multiplication of two n -bit numbers
always $O(n^2)$?

Multiply: Karatsuba

$$xy = 2^n x_L y_L + 2^{\frac{n}{2}} (x_L y_R + x_R y_L) + x_R y_R$$

We note that: $x_L y_R + x_R y_L = (x_L + x_R)(y_L + y_R) - x_L y_L - x_R y_R$

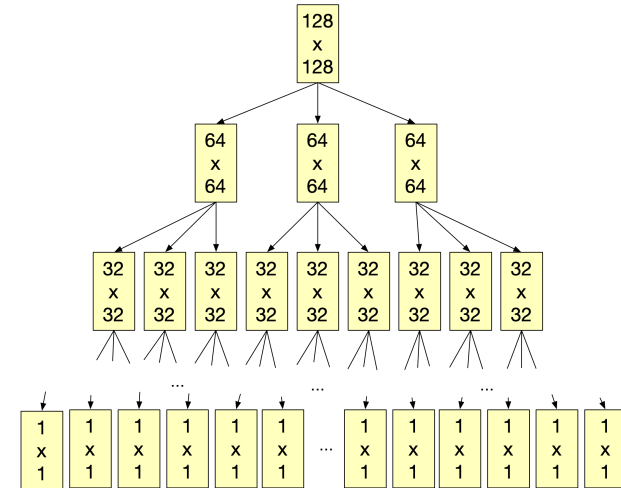
Only three multiplications: $x_L y_L, x_R y_R, (x_L + x_R)(y_L + y_R)$

New recurrence: $T(n) = 3T(\frac{n}{2}) + O(n), a = 3, b = 2, d = 1$

$\log_b a > 1$ so:

$$O(n^{\log_2 3}) \approx O(n^{1.585})$$

```
1 define mult3( $x, y$ ):  
2   if  $x == 0$ , return zero  
3   if  $x == 1$ , return  $y$   
4    $x_L, x_R := \text{split}(x), y_L, y_R := \text{split}(y)$   
5    $LL := \text{mult3}(x_L, y_L)$   
6    $RR := \text{mult3}(x_R, y_R)$   
7    $PS := \text{mult3}(x_L + x_R, y_L + y_R)$   
8   return  $2^n \cdot LL + 2^{\frac{n}{2}} (PS - LL - RR) + RR$ 
```



Most big int libraries use:

- Hardware for $n \leq 64$
- Toom-Cook (similar to Karatsuba) for $n \leq 4,000$
- Schönhage–Strassen (FFT-based) for $n > 4,000$

```
#include <gmp.h>
char inStr[1024] = "392...511";
mpz_t n;
mpz_init(n);
mpz_set_ui(n, 0);
mpz_set_str(n, inStr, 10);
mpz_mul(n, n, n); /* n = n * n */
mpz_out_str(stdout, 10, n);
```

$$x = [x_n, x_{n-1}, \dots, x_1]$$

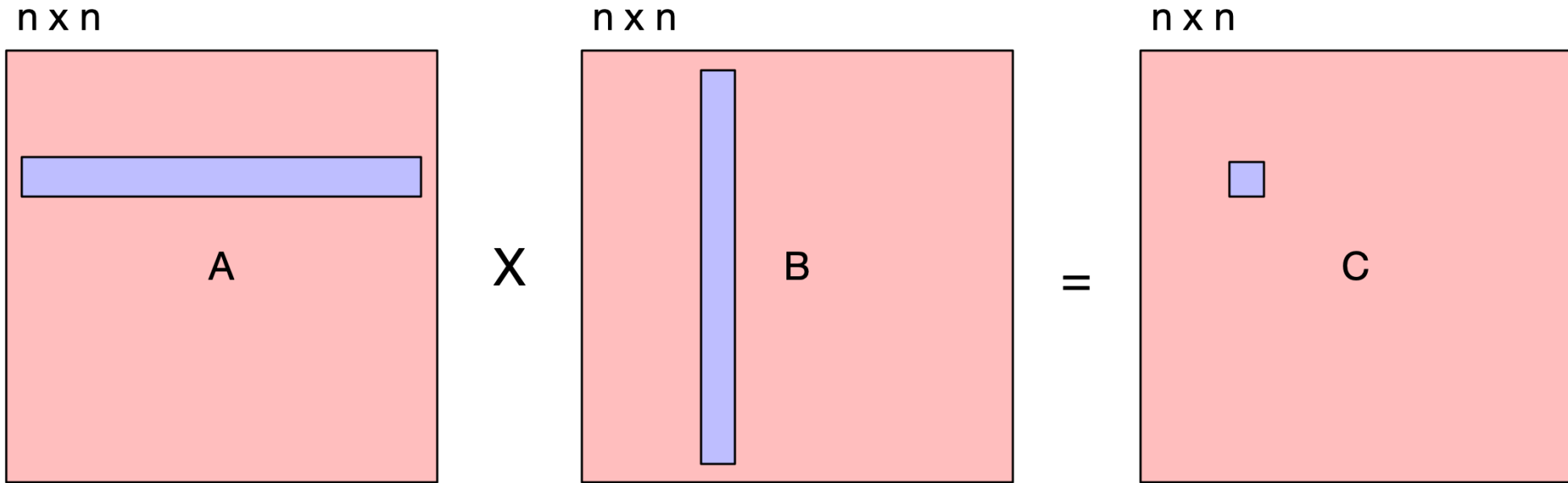
$$A^x = A^{[2^{n-1}x_n]} A^{[2^{n-2}x_{n-1}]} \dots A^{x_1}$$

Example: $3^{13} = 3^{[1,1,0,1]} = (3^{2^3})(3^{2^2})(3^0)(3^{2^0}) = (6,561)(81)(3) = 1,594,323$

```
1 define exp(A, x):  
2   result := I  
3   factor := A  
4   x_stack := bits of x (LSB at top)  
5   while bit := x_stack.pop():  
6     if bit:  
7       result := result * factor  
8       factor := factor × factor  
9   return result
```

Naive Matrix Multiplication

13/28



$O(n^3)$

Strassen Algorithm (1969)

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \quad A \times B = \begin{pmatrix} M_1 + M_4 - M_5 + M_7 & M_3 + M_5 \\ M_2 + M_4 & M_1 - M_2 + M_3 + M_6 \end{pmatrix}$$

where

$$M_1 := (A_{11} + A_{22}) \times (B_{11} + B_{22})$$

$$M_2 := (A_{21} + A_{22}) \times B_{11}$$

$$M_3 := A_{11} \times (B_{12} - B_{22})$$

$$M_4 := A_{22} \times (B_{21} - B_{11})$$

$$M_5 := (A_{11} + A_{12}) \times B_{22}$$

$$M_6 := (A_{21} - A_{11}) \times (B_{11} + B_{12})$$

$$M_7 := (A_{12} - A_{22}) \times (B_{21} + B_{22})$$

$$T(n) = 7T\left(\frac{n}{2}\right) + O(n^2), \text{ so } O(n^{\log_2 7}) \approx O(n^{2.8})$$

Wednesday's story: "Google's AlphaEvolve has found a faster way to multiply 4×4 matrices!"

Hmm.

Meet AlphaEvolve, the Google AI that writes its own code—and just saved millions in computing costs



Credit: VenturaBeat made with Midjourney

All of Fast Fourier Transforms!?

Evaluating a Polynomial

Given a polynomial $A = a_0 + a_1x + a_2x^2 + \dots + a_{n-1}x^{n-1}$

Naive evaluation of $A(x_0)$ is $O(n^2)$:

$$A(x_0) = a_0 + a_1x_0 + a_2x_0^2 + \dots + a_{n-1}x_0^{n-1}$$

Smarter way is $O(n)$:

$$A(x_0) = a_0 + x(a_1 + x(a_2 + x(a_3 + \dots + xa_{n-1})\dots))$$

Adding Polynomials

$$A(x) + B(x)$$

$$= a_{n-1}x^{n-1} + \dots + a_0 + b_{n-1}x^{n-1} + \dots + b_0$$

$$= (a_{n-1} + b_{n-1})x^{n-1} + \dots + (a_0 + b_0)$$

Assuming scalar addition is $O(1)$, polynomial addition is $O(n)$

Multiplying Polynomials

$$A(x)B(x)$$

$$= (a_{n-1}x^{n-1} + a_{n-1}x^{n-1} + \dots + a_0)(b_{n-1}x^{n-1} + b_{n-2}x^{n-2} + \dots + b_0)$$

$$= c_{2(n-1)}x^{2(n-1)} + \dots + c_0$$

$$\text{where } c_k = a_0b_k + a_1b_{k-1} + \dots + a_kb_0 = \sum_{i=0}^k a_ib_{k-i}$$

This is $O(n^2)$. Can we do it faster?

Any n points define the polynomial of degree $n - 1$

What if instead of multiplying the coefficients, we:

1. Pick n points
2. Evaluate A and B at those points
3. Multiply those results to get C at those points
4. Did interpolation to figure out the coefficients of C

Can we chose those n points such that steps 2 and 4 were crazy-fast?

Polynomial Divide and Conquer

21/28

Divide a polynomial A into its even and odd terms:

$$A(x) = 9x^5 + 3x^4 - 2x^3 + 5x^2 + 7x - 4$$

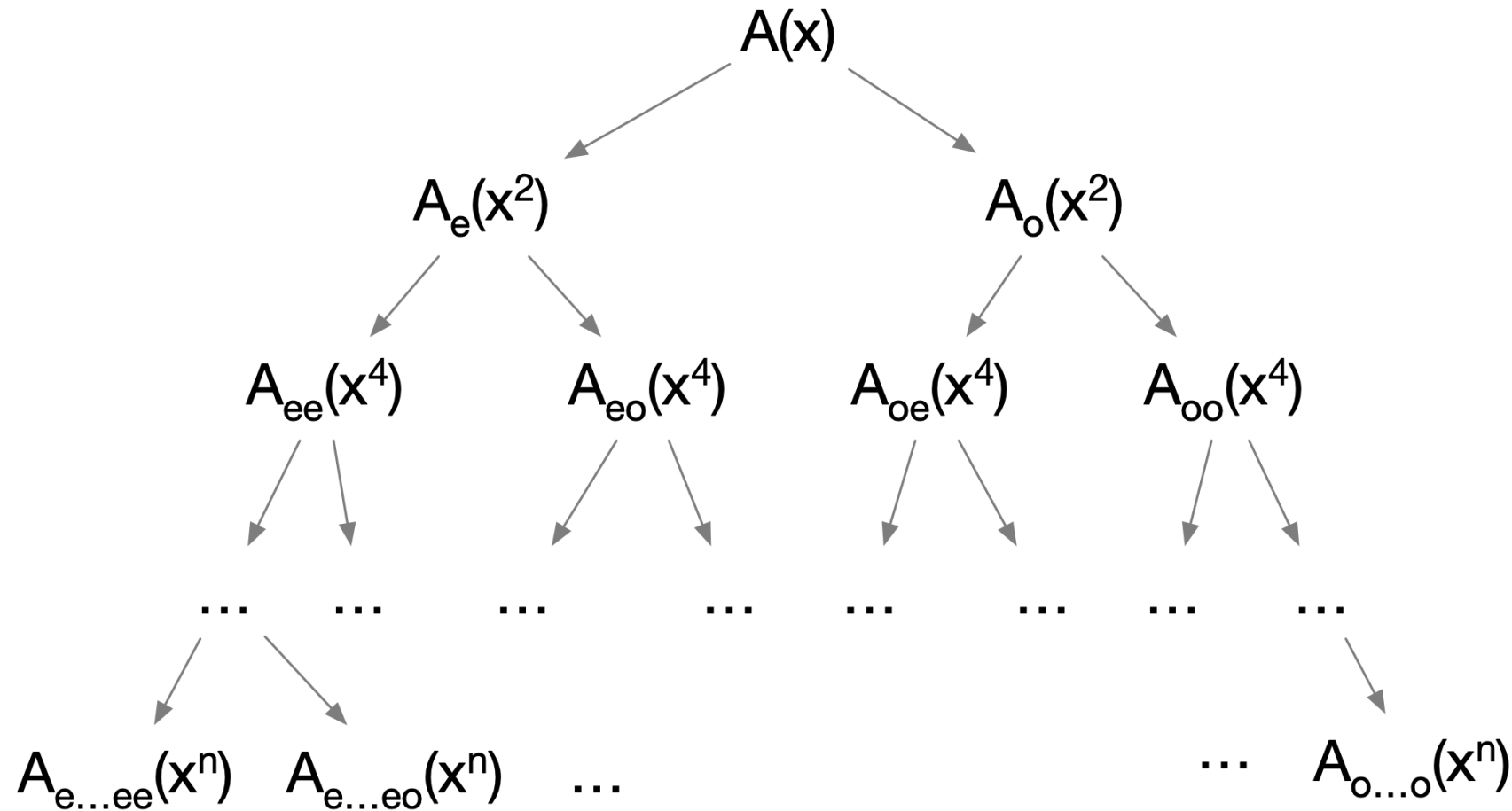
$$A_{e(x)} = 3x^2 + 5x - 4 \qquad A_{o(x)} = 9x^2 - 2x + 7$$

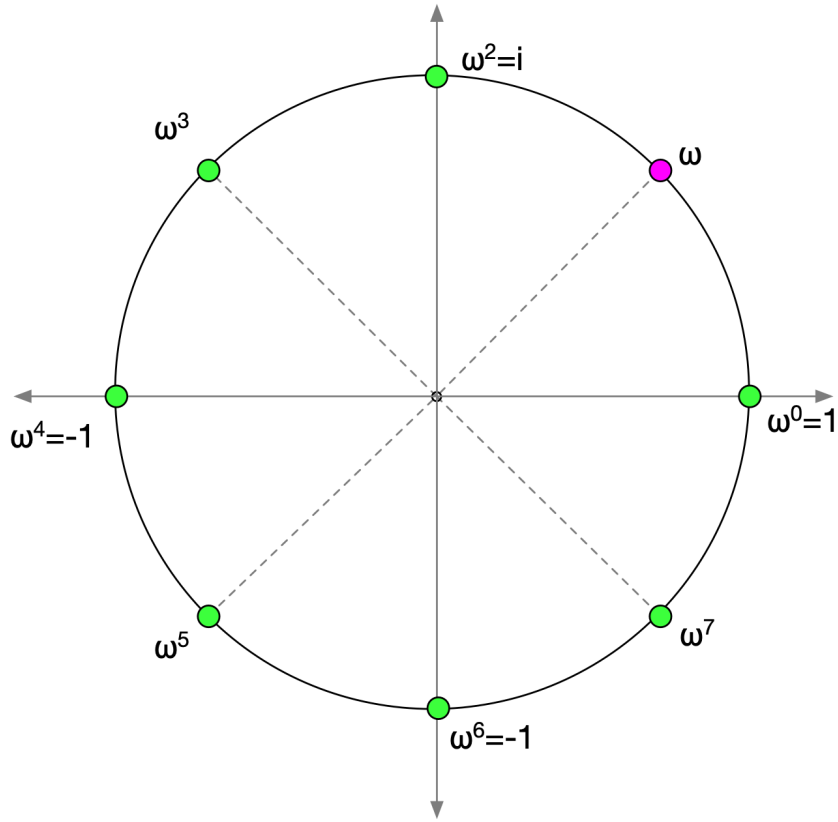
$$A(x) = A_e(x^2) + xA_o(x^2)$$

If I've already evaluated A at $x = 1$, what would be a really easy second value to evaluate?

Repeated Divide-and-Conquer

22/28

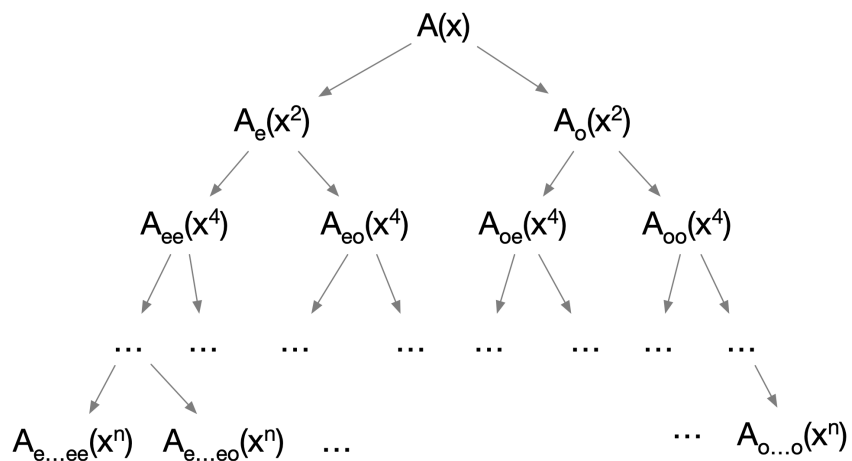




What is $\{x \mid x^8 = 1\}$?

$$\omega^k = e^{\frac{2k\pi i}{n}} = \cos \frac{2k\pi}{n} + i \sin \frac{2k\pi}{n}$$

where $k = 0, \dots, n - 1$



At top: Evaluate A of degree $n - 1$ at n points

Recurrence: Evaluate A of degree $\frac{n}{2} - 1$ at $\frac{n}{2}$ points

Do n multiplications and n additions: $O(n^1)$

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

FFT is $O(n \log n)$

FFT Pseudocode

Given:

- A : a polynomial of degree $\leq n - 1$
- ω an n th root of unity

```
1 define FFT( $A, \omega$ ):  
2   if  $\omega == 1$ : return  $A(1)$   
3   express  $A(x)$  in the form  $A_e(x^2) + xA_o(x^2)$   
4   evens := FFT( $A_e, \omega^2$ ) to evaluate  $A_e$  at powers of  $\omega^2$   
5   odds := FFT( $A_o, \omega^2$ ) to evaluate  $A_o$  at powers of  $\omega^2$   
6   for  $j$  in 0 to  $\frac{n}{2} - 1$ :  
7      $\text{result}[j] := \text{evens}[j] + \omega^j \text{ odds}[j]$   
8      $\text{result}[j + \frac{n}{2}] := \text{evens}[j] - \omega^j \text{ odds}[j]$   
9   return result
```

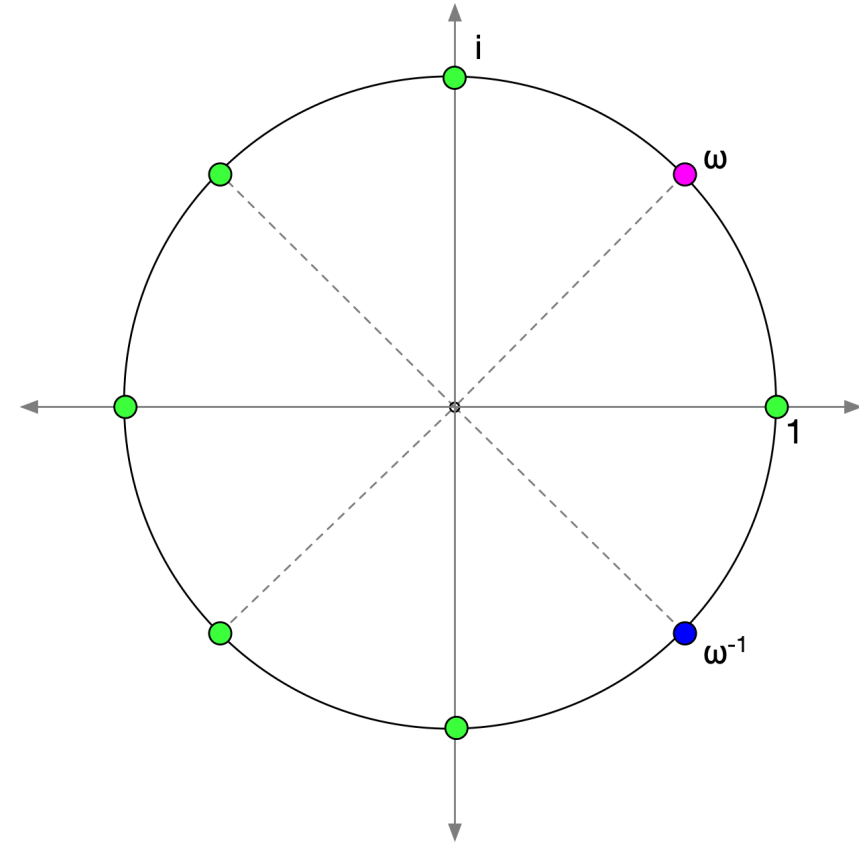
And back again...

Now we have a way to go from n coefficients to n values in $O(n \log n)$:

$$\text{values} := \text{FFT}(\text{coefficients}, \omega)$$

The FFT also does the inverse. Given values at powers of ω :

$$\text{coefficients} := \frac{1}{n} \text{FFT}(\text{values}, \omega^{-1})$$



The whole trip

Example: A and B be two polynomials of degree 7:

- Give them 16 coefficients by adding zero coefficients.
- Calculate the 16th root of unity: $\omega := \cos \frac{2\pi}{16} + i \sin \frac{2\pi}{16}$
- $\text{values}_a := \text{FFT}(A, \omega)$
- $\text{values}_b := \text{FFT}(B, \omega)$
- Element-wise multiplication gets us values_c
- $\omega^{-1} := \cos \frac{2\pi}{16} - i \sin \frac{2\pi}{16}$
- $C := \text{FFT}(\text{values}_c, \omega^{-1})$

Questions?

Slides by Aaron Hillegass